

Programmeringskurs för STORIA 1997

Föreläsning 12

Ändrad
24 Sep
16:08

Denna lektion är full av små program som demonstrerar hur man använder pekare i C. Så sakta som möjligt vill jag i minimala munsbiter introducera er för pekare.

Adresser, pekare

Variabler lagras i datorns minne. Minnet består av celler om en byte. Varje minnescell -- varje byte -- har en adress. Adressen är ett heltal -- "I C är allting heltal".

Operatören `&` plockar fram adressen till en variabel.

Operationen `sizeof()` ger en variablers storlek i byte.

Följande program deklarerar sex variabler. Deras storlekar och adresser skrivs ut.

```

1  /*
2      Exempel 19, PT DING+EING 1997, 1997-9-21
3
4      Nyheter:
5          - minnesadresser
6          - sizeof
7  */
8
9  #include <stdio.h>
10
11
12 void main()
13 {
14     double x;
15     float y;
16     int i;
17     short int j;
18     char c;
19
20     printf("Variabel Storlek Adress\n");
21     printf("-----\n");
22     printf("  c      %d byte %lu \n", sizeof(c), &c);
23     printf("  j      %d byte %lu \n", sizeof(j), &j);
24     printf("  i      %d byte %lu \n", sizeof(i), &i);
25     printf("  y      %d byte %lu \n", sizeof(y), &y);
26     printf("  x      %d byte %lu \n", sizeof(x), &x);
27 }
```

En körning ger:

```

1  Variabel Storlek Adress
2  -----
3      c      1 byte 4026530309
4      j      2 byte 4026530310
5      i      4 byte 4026530312
6      y      4 byte 4026530316
7      x      8 byte 4026530320
```

Nästa gång jag kör programmet kan variablerna hamna på andra adresser. Om du kör programmet på PC hamnar variablerna troligen på helt andra adresser.

Min dator valde att lagra variablerna *konsekutivt* -- i tur och ordning i minnesceller efter varandra.

Datorn har fyllt på sitt tilldelade minnesutrymme från

(slutet -- den först deklarerade variabeln hamnade sist.)

Pekare

Följande program skriver ut värdet på två olika variabler -- utan att nämna dem vid namn. Vi skriver nämligen ut dem via en *pekare*.

En pekare är en variabel som har en *adress* som värde. Adressen går vanligen till en variabel.

Pekare i C är *typade*. Varje pekare pekar endast ut en typ av värde. Man säger till exempel "pekare på int", eller "pekare på float". Eller kort och gott "float-pekare".

Typen bestäms i deklarationen. En stjärna *** anger att variabeln är en pekare.

Följande program deklarerar två *int*-variabler, samt en *int*-pekare.

```
1  #include <stdio.h>
2
3  void main()
4  {
5      int i = 12345, j = 54321;
6      int *p;
7
8      p = &i;
9      printf("i har adress %lu  \n", p);
10     printf("   och värde %d  \n\n", *p);
11
12     p = &j;
13     printf("j har adress %lu  \n", p);
14     printf("   och värde %d  \n\n", *p);
15 }
```

Operatorm *&* omvandlar från variabel till adress. Operatorm *** omvandlar från adress till variabel. Körningen ser ut som följer:

```
1  i har adress 4026530324
2      och värde 12345
3
4  j har adress 4026530320
5      och värde 54321
```

Notera alltså att vi hittar *i* och *j* via *p*. Vi hittar både adress och värde.

Ändra variabler via pekare

Allt vi kan göra med en variabel *x*, kan vi också göra via en pekare till *x*. Exempelvis ändra värdet på *x*:

```
1  #include <stdio.h>
2
3  main()
4  {
5      double x, *ptr;
6
7      ptr = &x;
8      x = 7;
9      printf("x = %lg\n", x);
```

```

10     *ptr = 8;
11     printf("x = %lg\n", x);
12 }

```

Körningen korroboreerar att det fungerar:

```

1  x = 7
2  x = 8

```

Inläsning via pekare

Före varje variabel i anropet till `scanf()` brukar vi skriva `&`. Det är adressoperatoren.

Funktionen `scanf()` vill ha adresser som parametrar. Ty `scanf()` vill ändra innehållet i variablerna, och då måste `scanf()` få veta var i minnet de är lagrade.

I nästa exempel sätts `ptr` till adressen till `i`. För att vi ska kunna läsa in till `i` via `ptr`.

Men `ptr` ska *inte* ha `&` framför sig -- eftersom `ptr` redan är en adress! Eller hur?

Tänk efter! Det stämmer!

```

1  #include<stdio.h>
2
3  main()
4  {
5      int i, *ptr;
6
7      ptr = &i;
8
9      printf("Mata in i: ");
10     scanf("%d", ptr); /* AHA! Inget "&"! */
11
12     printf("i = %d\n", i);
13 }

```

Ser bra ut:

```

1  Mata in i: 114422
2  i = 114422

```

Nyttan av pekare: ändra parametervärde

Hittills har jag bara gett meningslösa exempel. Bara för att visa hur pekare och adresser fungerar. Men man klarar sig faktiskt inte utan att använda pekare i programspråket C. Till exempel om en funktion ska ändra värdet på en variabel.

För att kunna ändra värdet måste funktionen veta var i minnet variabeln är lagrad -- vilken minnesadress den ligger på. Då måste vi använda en pekare som parameter -- precis som `scanf()`.

Nästa exempel: funktionen `halvera()` halverar sitt argument. Parametern `ptr` är en `double`-pekare.

Med hjälp av operatoren `*` kommer `ptr` åt variabeln den pekar på.

```

1  #include <stdio.h>
2
3  void halvera(double *ptr)
4  {
5      *ptr = *ptr / 2;

```

```

6  }
7
8  main()
9  {
10     double x = 1;
11
12     do {
13         printf("x = %g\n", x);
14         halvera(&x);
15     } while (x > 0.1);
16 }

```

Huvudprogrammet anropar `halvera()` tills `x` är mindre än `0.1`.

Huvudprogrammet ger adressen till `x` som argument till `halvera()`. För precis som `scanf()` vill `halvera()` ha adresser som parametrar.

```

1  x = 1
2  x = 0.5
3  x = 0.25
4  x = 0.125

```

`getfloat()` och `scanfloat()`

I följande exempel har jag implementerat funktionerna `getfloat()` och `scanfloat()`. De gör bägge samma sak -- läser in ett flyttal. Så här ser det ut när man kör:

```

1  Mata in två flyttal: 1.23 4.14
2  x = 1.23, y = 4.14

```

Här är källkoden:

```

1  #include <stdio.h>
2
3
4  float getfloat()
5  {
6      float x;
7      scanf("%f", &x);
8      return x;
9  }
10
11
12 void scanfloat(float *fp)
13 {
14     scanf("%f", fp);    /* AHA! Inte & framför fp! */
15 }
16
17
18 void main()
19 {
20     float x, y;
21
22     printf("Mata in två flyttal: ");
23     x = getfloat();
24     scanfloat(&y);
25
26     printf("x = %g, y = %g \n", x, y);
27 }

```

- Funktionen `getfloat()` innehåller inget nytt. Den returnerar ett värde som den erhållit av `scanf()` på de vanliga sättet. Det är heller inget konstigt med anropet av `getfloat()` på rad 23.

- Funktionen `scanfloat()` tar en adress som parameter (rad 12). En adress till en `float`. Parametern heter `fp`.

Parametern `fp` är en pekare på `float`.

Variabeln `y` är en `float`. Adressen till `y` är `&y`. Alltså anropar vi som så: `scanfloat(&y)` (rad 24).

Formella parametern `fp` får adressen till `y` som värde.

Därför behövs inget `&` i anropet till `scanf()` (rad 14).

Funktionen `scanf()` vill nämligen ha adresser som parametrar och `fp` är ju en adress! Nämligen adressen till `y`!

Ska vi byta värde med varann?

Så här kan `inte` `x` och `y` byta värden med varandra:

```
x = y;
y = x;
```

Bägge får `y`s gamla värde.

Om två variabler ska byta värde med varandra behövs en extra variabel. Den lagrar temporärt det ena värdet:

```
1 #include <stdio.h>
2
3 main()
4 {
5     char x = 'x', y = 'y', dummy;
6
7     printf("x = '%c', y = '%c' \n", x, y);
8
9     dummy = x;
10    x = y;
11    y = dummy;
12
13    printf("x = '%c', y = '%c' \n", x, y);
14 }
```

Körningen visar att det fungerar:

```
1 x = 'x', y = 'y'
2 x = 'y', y = 'x'
```

swapchar()

Det är lämpligt att definiera en funktion som byter värden på två variabler:

```
1 #include <stdio.h>
2
3 void swapchar(char *x, char *y)
4 {
5     char dummy;
6     dummy = *x;
7     *x = *y;
8     *y = dummy;
9 }
10
11 main()
12 {
13     char x = 'x', y = 'y', dummy;
14
15     printf("x = '%c', y = '%c' \n", x, y);
16     swapchar(&x, &y);
17     printf("x = '%c', y = '%c' \n", x, y);
18 }
```

Funjerar:

```
1 x = 'x', y = 'y'
```

```
2 x = 'y', y = 'x'
```

Nästa lektion

